

CTCC Orchestrator Documentation

1 Introduction

The `ctcc.py` script is the CLI entry point for the Comprehensive Transmission Cost Calculator (CTCC). It orchestrates all cost and benefit calculation modules by running them as subprocesses in a fixed order, sets environment variables for input/output mode (YAML/JSON, CSV/JSON) and optional flags, and aggregates results. In CSV mode it updates `batch_summary.csv` and runs the BCR calculator; in JSON mode it aggregates per-script JSON outputs, computes BCR from the aggregated data, and writes a single `ctcc_results_<id>.json`.

2 Method Overview

2.1 Purpose

1. Parse command-line arguments (e.g. `--no_emissions`, `--capital_only`, `--simple`).
2. Load Primary BCR config from YAML or JSON (based on `CTCC_INPUT_MODE`) and override args with config flags where present.
3. Build `BCRConfig` from args and set env vars (e.g. `CTCC_NO_CONGESTION`, `CTCC_NO_WF_LIABILITY`).
4. Generate or use existing `CTCC_SCENARIO_ID`.
5. Build the list of scripts to run (prerequisites first, then capital costs, then optional modules based on flags; `--capital_only` runs only `weighted_miles`, `build_costs`, `row_costs`, `environmental_mitigation`).
6. Run each script as a subprocess from `scripts/` with the current environment; track successes and failures.
7. If output mode is CSV: run `calculate_and_display_bcr` on `batch_summary.csv`, then add BCR metrics to `batch_summary.csv` via `CSVOutputManager`.
8. If output mode is JSON: `aggregate_json_outputs` loads and merges `json_output_<id>_*.json` files, `build_bcr_data_from_json` extracts a flat dict from aggregated results, BCR is computed via `calculate_benefits/calculate_costs/calculate_bcr_metrics`, then `write_final_json_output` writes `ctcc_results_<id>.json` with BCR and summary.
9. Exit with code 1 if any script failed.

2.2 Calculation Approach

Orchestration only; no cost/benefit formulas are implemented in `ctcc.py`. Calculations live in the scripts in `scripts/`. Sequencing and mode handling (env vars, script list, BCR aggregation) are the only logic here.

2.3 Inputs and Outputs

Inputs:

- Command-line flags: `--norisk`, `--simple`, `--no_wf_liability`, `--no_emissions`, `--no_line losses`, `--capital_only`, `--no_oandm`, `--no_insurance`, `--no_delay_costs`, `--no_wildfire`, `--no_outages`, `--no_congestion`, `--no_curtailment`.
- Environment variables: `CTCC_INPUT_MODE`, `CTCC_OUTPUT_MODE`, `CTCC_JSON_DATA_FILE`, `CTCC_SCENARIO_ID` (optional; auto-generated if unset).
- Primary BCR config: loaded from YAML or JSON (via `load_primary_bcr_config`) when available; overrides the above flags for which modules run.

Outputs:

- CSV mode: `outputs/batch_summary.csv` (and other CSVs written by scripts), updated with BCR metrics; BCR printed to stdout.
- JSON mode: `outputs/ctcc_results-<scenario_id>.json` containing aggregated costs, benefits, technical parameters, summary, and BCR; intermediate `json_output-<id>_*.json` files are removed after merge.
- Exit code 0 if all scripts succeeded, 1 if any failed.

2.4 Integration with Other Scripts

`ctcc.py` runs (as subprocesses) and/or imports:

- `scripts/weighted_miles.py`, `build_costs.py`, `row_costs.py`, `environmental_mitigation.py`, `revenue.py`, `insurance_costs.py`, `delay_costs.py`, `wildfire_costs.py`, `outage_costs.py`, `congestion_curtailment_reduction.py`, `energy_losses.py`, `oandm.py`, `emissions.py`, `line_loss_costs.py` (subset depending on flags).
- `scripts/bcr_calculator.py` — `calculate_and_display_bcr`, `calculate_benefits`, `calculate_costs`, `calculate_bcr_metrics`.
- `scripts/csv_output_manager.py` — `CSVOutputManager`, `BATCH_SUMMARY_FIELDS`.
- `scripts/json_output_manager.py` — `JSONOutputManager` (used in `aggregate_json_outputs`).
- `scripts/json_loaders.py` or `scripts/yaml_loaders.py` — `load_primary_bcr_config` (depending on `CTCC_INPUT_MODE`).

3 Key Functions

3.1 `run_script(script_name, quiet=False)`

Runs a single Python script from the `scripts/` directory as a subprocess with the current environment.

Parameters:

- `script_name` (str): Name of the script (e.g. `weighted_miles.py`).
- `quiet` (bool): If True, suppress success/failure messages.

Returns: True if the script exited with code 0, False otherwise.

Key logic:

```
1 env = os.environ.copy()
2 result = subprocess.run(
3     [sys.executable, script_name],
4     capture_output=True,
5     text=True,
6     cwd="scripts",
7     env=env,
8 )
9 return result.returncode == 0
```

3.2 `build_bcr_data_from_json(json_results)`

Builds a flat dictionary suitable for BCR calculations from the aggregated JSON results (costs and benefits by category).

Parameters: `json_results` (dict): Aggregated results from `JSONOutputManager.get_json_results()`.

Returns: Dict with keys such as `congestion_benefit_pv`, `build_cost_pv`, `delay_cost_pv`, etc., used by `calculate_benefits` and `calculate_costs`.

Key logic:

```
1 costs = json_results.get("costs", {}) or {}
2 benefits = json_results.get("benefits", {}) or {}
3 congestion = benefits.get("congestion_curtailment", {}) or {}
4 build = costs.get("build", {}) or {}
5 # ... extract PV and nominal fields from each category
6 return {
7     "congestion_benefit_pv": congestion.get("congestion_benefit_pv",
8         0) or 0,
9     "build_cost_pv": build.get("total_pv", 0) or 0,
10    # ... all BCR inputs
11 }
```

3.3 aggregate_json_outputs(scenario_id, output_dir="outputs")

Finds all `json_output_<scenario_id>*.json` files in `output_dir`, loads each into a `JSONOutputManager` aggregator, merges results, and deletes the intermediate files.

Parameters:

- `scenario_id` (str): Scenario ID used in filenames.
- `output_dir` (str): Directory containing the JSON output files.

Returns: The `JSONOutputManager` instance with merged results (call `.get_json_results()` for the dict).

Key logic:

```
1 pattern = os.path.join(output_dir, f"json_output_{scenario_id}*.  
    json")  
2 json_files = glob.glob(pattern)  
3 aggregator = JSONOutputManager(scenario_id=scenario_id)  
4 for json_file in json_files:  
5     aggregator.load_from_file(json_file)  
6     os.unlink(json_file) # remove intermediate file  
7 return aggregator
```

3.4 write_final_json_output(aggregator, bcr_results, scenario_id, output_dir, csv_equivalent, summary_override)

Adds BCR metrics to the aggregator, computes summary, and writes `ctcc_results_<scenario_id>.json`.

Key logic:

```
1 if bcr_results:  
2     aggregator.add_bcr_metrics(bcr_results)  
3 aggregator.calculate_summary()  
4 results = aggregator.get_json_results()  
5 if csv_equivalent is not None:  
6     results["csv_equivalent"] = csv_equivalent  
7 if summary_override is not None:  
8     results["summary"] = summary_override  
9 output_file = os.path.join(output_dir, f"ctcc_results_{scenario_id}.  
    json")  
10 with open(output_file, "w") as f:  
11     json.dump(results, f, indent=2)  
12 return output_file
```

3.5 main()

Parses arguments, loads BCR config, sets env vars, builds script list, runs each script via `run_script`, then either (CSV) runs `calculate_and_display_bcr` and updates `batch_summary.csv` or (JSON) aggregates JSON outputs, computes BCR from aggregated data, and calls `write_final_json_output`. Exits with 1 if any script failed.

4 Example

Default run (YAML input, CSV output, all modules):

```
1 python ctcc.py
```

JSON input and JSON output with scenario ID `test`:

```
1 export CTCC_INPUT_MODE=json
2 export CTCC_OUTPUT_MODE=json
3 export CTCC_JSON_DATA_FILE=server/json/final_combined.json
4 export CTCC_SCENARIO_ID=test
5 python ctcc.py
```

Capital-only run (weighted miles, build, ROW, environmental mitigation):

```
1 python ctcc.py --capital_only
```

Simple output (only BCR analysis printed):

```
1 python ctcc.py --simple
```